$ specship · for developers

The software factory for Claude Code

Map the code. Contract the intent. Build, prove and watch every requirement — one line, on your machine.

SpecShip v0.27 · @specship/specship · MIT

THE LINE

Six stations from intent to proven code

01
Map
Index the repo into a code graph

02
Spec
Intent as testable criteria

03 · GATE
Plan
You approve or reject

04
Build
In an isolated worktree

05
Prove
Tests plus an independent critic

06
Watch
Drift alerts and CI gates

Deterministic tooling at every station; a human at the two gates that matter — the plan and the final diff.

SET UP

Three commands and a restart

```
# 1 · get the CLI (Node 22.5–24)
npm i -g @specship/specship@latest
# 2 · wire Claude Code + index the repo
specship install
# 3 · check runtime, FTS5, MCP, index
specship doctor
```

Install writes

- MCP server entry
- `/specship:*` commands
- spec-author skill + steering
- hooks, status line, permissions

Flags: `--no-sdd` retrieval only · `--with-jira` · `--with-designer`

STATION 01 · THE MAP

A deterministic code graph, not a summary



Local. Lives in `.specship/`; no network, no telemetry.

Fresh. File watcher + git hooks; the index trails edits by about a second.

Honest. Unsure edges stay silent rather than guess.

MCP TOOLS

Pick the tool by the question

You ask	Claude calls
How does X work? Trace X → Y	`specship_explore`
Where is X? Full source of X	`specship_search · _node`
Who calls it? What does it call?	`specship_callers · _callees`
What breaks if I change it?	`specship_impact`
What's in this directory?	`specship_files`
Healthy? Breaks our rules?	`specship_health`
What does REQ-X say? Status?	`specship_spec`
Index ready? Which version?	`specship_status`

Target: a flow question answered in one `explore` call, zero Read/Grep.

SLASH COMMANDS

Six doors cover the daily flow

Command	What it opens
`/specship:explore`	Read code: a region, a flow, a change’s blast radius
`/specship:spec`	Intent: idea, new, fast, design, implement, review
`/specship:check`	Gate, drift queue, fix / relink, code health
`/specship:day`	Your sprint issues → pick one → start
`/specship:jira`	Bind the board, choose an epic, pick work
`/specship:learn`	Turn a good session into a skill proposal

No argument opens the door’s default view — the funnel, the gate, the menu.

STATIONS 02 — 06

The spec is the contract. Code follows it.

Drift from the spec is a bug — and SpecShip tells you the second it happens.

STATION 02 · THE SPEC

Markdown with IDs, criteria and links

```
specs/auth.md
<!-- id: REQ-AUTH-001 -->
## Failed logins MUST be rate-limited
implementations:
- src/auth/login.ts:LoginService.attempt
## Acceptance
<!-- id: REQ-AUTH-001.A1 -->
- 6th failed attempt in 60 s is refused
<!-- id: REQ-AUTH-001.A2 -->
- A successful login resets the count
```

Stable IDs for every requirement and criterion

RFC 2119 keywords: MUST, SHOULD, MAY

path:Symbol links become real graph edges

In the repo, reviewed in PRs like code

AUTHORING

Describe it; the spec-author drafts it

`spec idea`

Capture a one-liner so it isn't lost

`spec new`

Draft from the graph; asks only the gaps

`spec fast`

Fast path for a small, clear change

`spec design`

Claude Design link → snapshot → spec

`spec review`

Quality pass: every criterion testable

`specship lint`

Deterministic checks; non-zero exit on errors

Domain facts — confirmed business terms and rules ride along with every spec lookup.

STATIONS 03 – 05 · IMPLEMENT

One command, two human gates

```
> /specship:spec implement REQ-AUTH-001
```



The critic grades each criterion **satisfied**, **partial** or **unaddressed**. Nothing merges without you.

STATION 06 · LINK STATE

Every link carries an honest state

State	Means
Implemented	Code claims the requirement. Not proven yet.
Verified ✓	A linked test ran and passed. The only “done”.
Drifted	Code or spec changed after linking. Re-check it.
Broken	Verification ran and failed.
Orphaned	The linked symbol was renamed or deleted.

Drift is automatic: the file watcher re-checks linked code on every edit, so a refactor flips the state in seconds.

PROOF

“Verified” needs test evidence

1 · Link

`implementations:` block, or `@implements` on the symbol

2 · Test

Write tests per criterion: `spec behaviour`

3 · Ingest

`specship verify` reads the vitest report, promotes or demotes

```
$ specship coverage REQ-AUTH-001
# per criterion: test-link state + verdict
```

No passing test, no Verified — an agent can’t self-award it.

QUALITY GATES · CI

The same checks on every commit

Command	What it does
`specship check`	Fails on drift, fitness, health or behaviour
`specship lint`	Fails on malformed specs or missing criteria
`specship fitness`	Fails on an architecture-rule violation
`specship suppressions`	Fails when the diff adds `@ts-ignore`, `noqa` and friends
`specship affected`	Lists only the tests your diff touches

Start advisory; switch gates to blocking one at a time.

AROUND THE LINE

One source of truth, four surfaces

Dashboard

`specship desktop · 127.0.0.1`

Traceability, drift queue, workflow approve/reject

JIRA

`opt-in · --with-jira`

Specs as stories, sprint coverage, regression packs

Designer

`experimental · --with-designer`

Claude Design project → snapshot → spec

Gemini CLI

`opt-in · --target gemini`

MCP tools only; no hooks or slash commands

A DAY ON THE LINE

Ticket to green PR in five moves

You run	You get
`/specship:day`	Pick and start [PROJ-142]
`/specship:spec new …`	REQ-REFUND-004, reviewed
`/specship:spec implement …`	Approve plan, review diff
`/specship:check`	Gate green, Verified ✓
`git push`	CI re-runs specship check

Weeks later a refactor touches refunds → **Drifted** → re-prove → Verified again.

WORKED EXAMPLE · 1/5 · ASK

Start with one command

```
> /specship:spec new "Lock out repeated failed logins"
● specship_explore LoginService.attempt
Two gaps before I draft:
  1. Count per account or per IP?
  2. What does the blocked attempt return?
> Per IP, 5 per minute. 429 with Retry-After.
✓ Wrote specs/auth-rate-limit.md · REQ-AUTH-001
```

The spec-author reads the real code first, then asks only what the graph can’t answer.

WORKED EXAMPLE · 2/5 · SPEC

The spec it writes — three testable criteria

```
specs/auth-rate-limit.md
<!-- id: REQ-AUTH-001 -->
## Failed logins MUST be rate-limited per IP
## Acceptance
- A1 6th failure in 60 s → 429 + Retry-After
- A2 A successful login resets the count
- A3 Other IPs are unaffected
(each criterion carries its own id comment)
```

```
$ specship lint specs/
✓ no errors
```

Review it like code. It's a markdown file in the PR.

No code linked yet — the spec exists before the implementation.

Approve the plan, get linked code

```
> /specship:spec implement REQ-AUTH-001
```

Plan · you approve

1. RateLimiter keyed by IP
2. Call it from LoginService.attempt
3. Reset on success
4. One test per criterion

src/auth/rate-limiter.ts

```
/** @implements REQ-AUTH-001 */
export class RateLimiter {
  check(ip: string): Verdict {
    if (this.fails(ip) >= 5)
      return blocked(this.resetIn(ip));
    return ALLOW;
  }
}
```

The `@implements` comment makes the link: REQ-AUTH-001 is now **Implemented**.

One test per criterion, then Verified

```
tests/auth/rate-limit.test.ts
/** @verifies REQ-AUTH-001.A1 */
function blocksSixthFailure() {
  fail(5, ip);
  const res = login(BAD, ip);
  expect(res.status).toBe(429);
  expect(res.retryAfter).toBeDefined();
}
it('blocks 6th failure', blocksSixthFailure);
// …same for A2, A3
```

```
$ npx vitest run --reporter=json \
    --outputFile=report.json
$ specship verify --report report.json
$ specship coverage REQ-AUTH-001
✔ REQ-AUTH-001 verified
  ✔ .A1 verified · tests:1
  ✔ .A2 verified · tests:1
  ✔ .A3 verified · tests:1
3 of 3 criteria have linked tests.
```

`@verifies` links the test as evidence; a passing run promotes it to **Verified**.

WORKED EXAMPLE · 5/5 · WATCH

Six weeks later, a “harmless” refactor

```
src/auth/rate-limiter.ts
- if (this.fails(ip) >= 5)
+ if (this.fails(ip) > 5)
```

```
$ specship drifted
⚠ REQ-AUTH-001 drifted
  RateLimiter.check changed
```

Verified ✓ → Drifted · edit seen → Broken · A1 fails → Verified · fixed

CI’s `specship check` fails the PR — the off-by-one never reaches production.

Map it. Spec it.
Ship it. Prove it.

Start with the map — no process change. Add specs and gates where proof matters most.

```
npm i -g @specship/specship@latest && specship install
```